

Model-Based Design and Verification of an Autonomous Electric Robotaxi (AER) Highway Merge System

Andrej Cvecka, Yunseo Cho, Natatas Homrossukon, Dominik Łuniewski, and
Patrick Schürpf

University of Southern Denmark (SDU), Odense, Denmark sdu@sdu.dk
<https://www.sdu.dk>

Abstract. This report presents the design, verification, and validation of a Cyber-Physical System (CPS) for an autonomous robotaxi service. In Student Group Activity I, state machine models, an activity diagram, and a use case diagram are used to analyze, design, and model an autonomous robotaxi service. In Student Group Activity II, based on the informal requirement elements from the first student group activity, the design is verified and validated using UPPAAL TA models, the verifier, and simulations in UPPAAL. Based on the results, possible errors and reasons for failures are identified.

1 Introduction

A robotaxi is an autonomous vehicle designed to transport passengers without direct human control. Passenger safety and system reliability are critical, the design and validation of robotaxi systems is needed. This report focuses on the design and evaluation of a robotaxi Cyber-Physical System CPS using Systems Modeling Language SysML. The objective of the report is to demonstrate how SysML modeling and V&V techniques are used in the development of reliable and safe autonomous transportation systems. The report models the core AI subsystem responsible for autonomous highway merging, a complex maneuver that requires continuous perception of the environment, decision-making, trajectory planning, and safety monitoring.

2 Requirements

2.1 Text-Based Requirements

The autonomous highway merge maneuver is a safety-critical operation requiring precise coordination between perception, prediction, and motion planning subsystems. To ensure both operational efficiency and passenger safety, the system must adhere to a strict set of functional and safety requirements.

These specifications guide the deterministic behavior of the vehicle's AI decision-making core during the high-stakes transition from an on-ramp to an active highway lane. A comprehensive breakdown of these engineering specifications, including their structural IDs and technical rationales, is detailed in the following table.

ID	Requirement	Rationale
MERGE-001	The AI shall continuously track all vehicles in the adjacent highway lane within a 150-meter range while on the on-ramp.	Ensures the system maintains awareness of traffic conditions before the merge point.
MERGE-002	The AI shall calculate relative velocities and predicted positions for each tracked vehicle over a 10-second horizon.	Enables predictive gap analysis rather than reactive decision-making.
MERGE-003	The AI shall identify a minimum safe gap of at least 4 seconds between the lead and following vehicles at the target merge speed.	Ensures safe following distances are maintained after merging.
MERGE-004	The AI shall compute and command a smooth acceleration profile to synchronize the vehicle's speed with the target gap's arrival at the merge point.	Prevents last-second speed adjustments and ensures comfortable, predictable merging.
MERGE-005	If no acceptable gap is identified before the final 50 meters of the merge lane, the AI shall initiate a "gap-request" (speed reduction + visual signaling).	Communicates intent to other drivers when autonomous gap selection fails.
MERGE-006	The AI shall abort a merge attempt and transition to the next best gap if the target gap closes below the minimum safe threshold.	Prioritizes safety over maneuver completion by using braking to avoid unsafe merges.
MERGE-007	In emergency situations where a collision is imminent, the AI shall override comfort limits and apply maximum braking to avoid or mitigate impact.	Ensures safety takes precedence over passenger comfort in critical scenarios.

Table 1. Text-based requirements for autonomous merge maneuver

2.2 Requirement Diagram

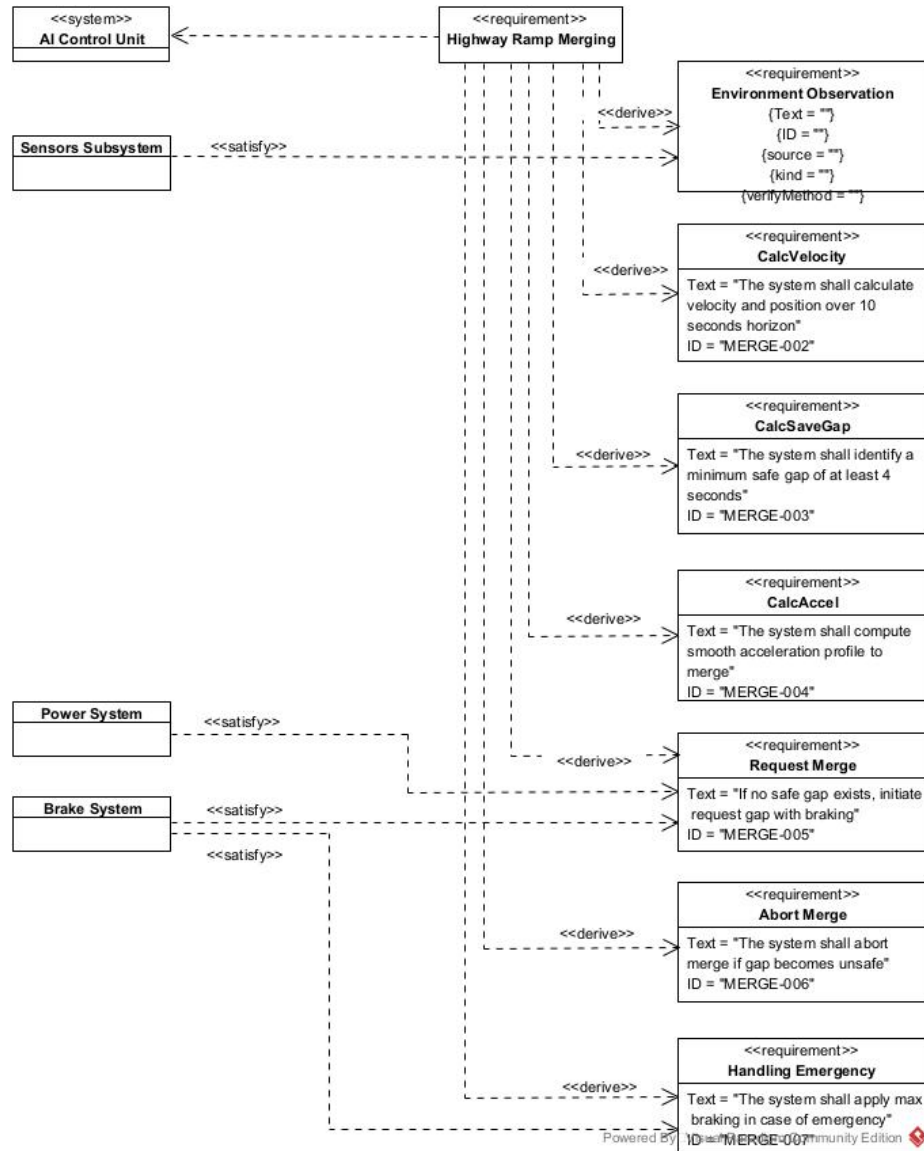


Fig. 1. Requirement Diagram

This Requirement Diagram illustrates a SysML requirement diagram for a highway merging scenario (req HWY Merge). It defines the relationships between

system architecture components (blocks, subsystems, systems) and the functional requirements necessary to safely execute a highway ramp merge.

The diagram is structured into three main layers:

1. **System Hierarchy (Left/Top):** The physical and digital breakdown of the vehicle.
2. **High-Level Requirement (Center/Top):** The primary functional goal.
3. **Derived Requirements and Satisfying Elements (Right/Bottom):** The specific, actionable requirements derived from the main goal and the components responsible for fulfilling them.

Detailed Diagram Breakdown

1. System Architecture and Structure

At the top left, the diagram establishes the structural hierarchy of the vehicle:

- «*System*» *Autonomous Electric Robotaxi*: The overarching system.
- «*subSystem*» *AI Control Unit*: Connected via a «contain» relationship, indicating that the AI Control Unit is a subcomponent housed within the Robotaxi. This unit is directly responsible for overseeing the Highway Ramp Merging capability.

2. High-Level Requirement

- «*requirement*» *Highway Ramp Merging*: This serves as the parent requirement. It acts as the root function for the entire subsystem during this specific operational design domain (ODD) scenario.

3. Derived Functional Requirement

The high-level merging capability is broken down into seven distinct, measurable, and lower-level requirements (IDs MERGE-001 through MERGE-007):

Requirement ID	Type / Title	Description
MERGE-001	Environment Observation	The system shall track vehicles in the adjacent lane.
MERGE-002	Calculations	The system shall calculate velocity and position over a 10 seconds horizon.
MERGE-003	Calculations	The system shall identify a minimum safe gap of at least 4 seconds.
MERGE-004	Calculations	The system shall compute a smooth acceleration profile to merge.
MERGE-005	Request Merge	If no safe gap exists, initiate a request gap with braking.
MERGE-006	Abort Merge	The system shall abort the merge if the gap becomes unsafe.
MERGE-007	Handling Emergency	The system shall apply max braking in case of an emergency.

4. Component Satisfaction and Support

The blocks on the left represent physical hardware components or subsystems that are mapped to these requirements to guarantee implementation via «*satisfy*» relationships:

- «*block*» *Sensors*: Satisfies MERGE-001 (Environment Observation). This indicates that the physical sensor suite (e.g., LiDAR, radar, cameras) is responsible for tracking adjacent vehicles.
- «*block*» *Power Systems*: Connects to the satisfaction path for executing the required velocity/acceleration behaviors.
- «*block*» *Brake Systems*: Directly satisfies MERGE-005 (initiating a gap with braking) and MERGE-007 (applying max braking during emergencies).
- «*block*» *Steering System*: Features a «*support*» relationship pointing upward to the Brake Systems. This highlights a functional dependency where steering and braking must cooperate or support one another to maintain vehicle stability during critical deceleration or maneuvering phases.

3 System Modeling

3.1 Use Case Description

The considered use case describes a Level 4 Autonomous Electric Robotaxi (AER) executing a highway merge maneuver from an on-ramp into high-speed traffic. This scenario represents a complex and safety-critical driving operation that requires continuous interaction among the vehicle’s sensing, decision-making, and control subsystems.

During the maneuver, the autonomous driving system continuously acquires and processes data from multiple onboard sensors to perceive the surrounding environment. The system identifies nearby vehicles, estimates their states, and predicts their future trajectories and driving intentions. Based on this information, the AI-based planning and control modules generate and execute a safe and efficient merging trajectory.

To complete the maneuver, the vehicle performs coordinated steering, braking, and propulsion control actions while maintaining compliance with safety and operational constraints. The scenario demonstrates the complexity of real-time autonomous driving and highlights the importance of Model-Based Systems Engineering (MBSE) in managing subsystem interactions, system integration, functional decomposition, and safety-critical requirements.

3.2 Block Definition Diagram (BDD)

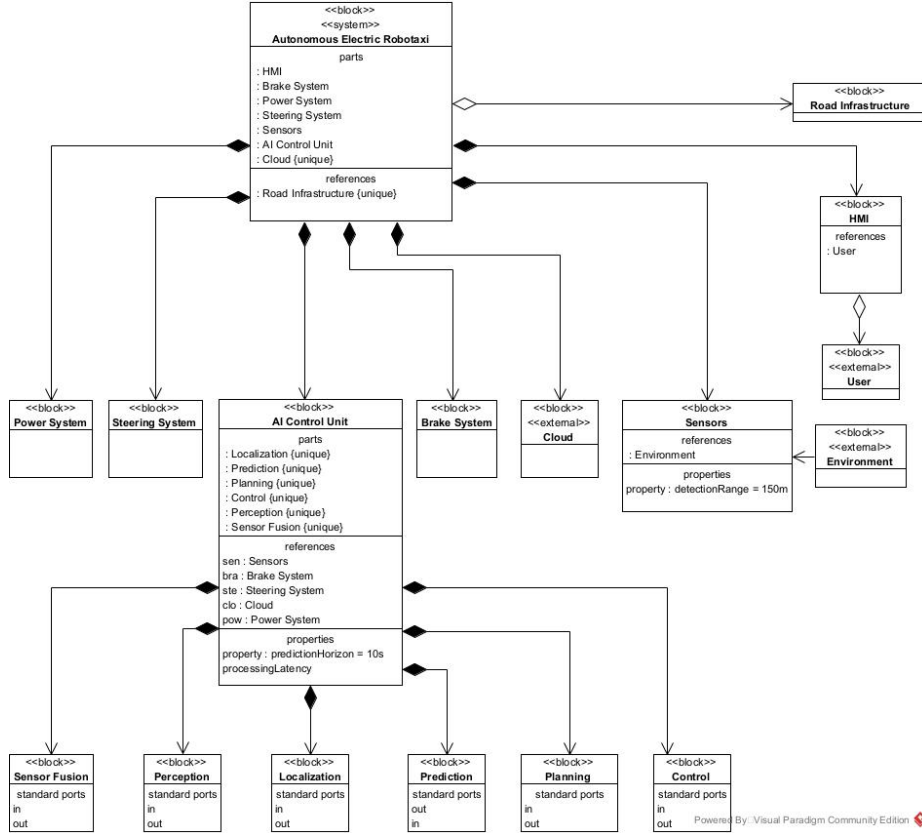


Fig. 2. Block Definition Diagram of the Autonomous Electric Robotaxi System Architecture

A Block Definition Diagram (BDD) in Systems Modeling Language (SysML) is used to represent the structural hierarchy and organization of a system. The BDD defines the system blocks, their properties, relationships, and subsystem composition, thereby providing a high-level representation of the overall system architecture. In MBSE, BDDs support system decomposition and facilitate the analysis of subsystem relationships and interfaces.

The Autonomous Electric Robotaxi system consists of several major subsystems, including the Power System, Steering System, Brake System, Sensor System, Cloud System, Human–Machine Interface (HMI), and AI Control Unit. In addition to these internal subsystems, external entities such as the User, En-

vironment, and Road Infrastructure interact with the vehicle through defined interfaces and reference relationships.

The AI Control Unit communicates with multiple vehicle subsystems, including the Sensor System, Brake System, Steering System, Power System, and Cloud infrastructure. The BDD also specifies key system properties, such as sensor detection range and prediction horizon, which are essential for autonomous driving performance and safety analysis. The overall structural relationships and subsystem composition are illustrated in Figure 2.

3.3 Internal Block Diagram (IBD)

An Internal Block Diagram (IBD) in SysML is used to describe the internal structure of a system and the interactions among its components. The IBD illustrates subsystem interfaces, ports, data flows, and communication pathways, making it particularly valuable in MBSE for analyzing subsystem integration, information exchange, and internal system behavior.

The AI Control Unit is considered the most safety-critical subsystem of the AER because it is responsible for autonomous perception, localization, prediction, planning, and vehicle control. As shown in Figure 3, the AI Control Unit is decomposed into the following primary functional components:

1. Sensor Fusion
2. Perception
3. Localization
4. Prediction
5. Planning
6. Control

Sensor data acquired from the Sensor System are first processed by the Sensor Fusion component, which integrates information from multiple sensing modalities into a unified representation of the surrounding environment. The Perception component subsequently interprets the environment and generates an environment model that is used by the Localization component to estimate the vehicle's position, orientation, and operational state.

Using the localized environment model, the Prediction component estimates the future trajectories and behavior of surrounding road users. Based on these predictions, the Planning component generates a safe and efficient trajectory for the highway merge maneuver. Finally, the Control component converts the planned trajectory into executable steering, braking, and propulsion control commands, which are transmitted to the Steering System, Brake System, and Power System.

In addition to vehicle control functions, operational and diagnostic data are transmitted to the Cloud System for monitoring, analytics, and reporting purposes. The internal architecture and data flow relationships of the AI Control Unit are illustrated in Figure 3.

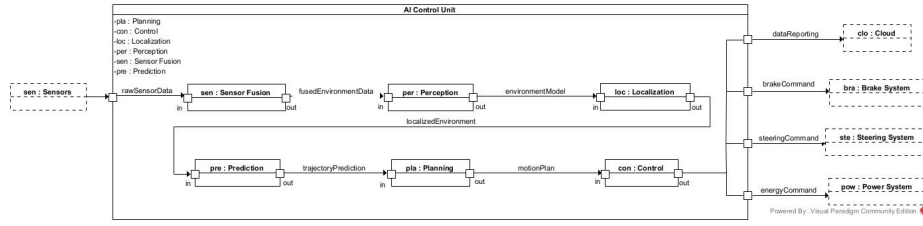


Fig. 3. Internal Block Diagram of the AI Control Unit and Data Flow Architecture

4 Process Description

4.1 Activity Diagram

The Activity Diagram provides a functional representation of the autonomous highway merge process and illustrates the sequence of actions performed by the AER during a merge maneuver. While the Block Definition Diagram (BDD) and Internal Block Diagram (IBD) describe the structural architecture of the system, the Activity Diagram focuses on the operational workflow and the interaction between the vehicle subsystems throughout the execution of the use case.

To improve clarity and subsystem traceability, the diagram is organized into swimlanes representing the primary functional domains of the system: the Vehicle, Perception System, Planning Module, Vehicle Control System, and Safety & Emergency System. Each swimlane contains the activities performed by the corresponding subsystem and highlights the flow of information and control between system components.

The process begins when the vehicle enters the highway on-ramp and approaches the merge area. The Perception System continuously monitors the surrounding environment by tracking nearby vehicles and predicting their future trajectories. The resulting environmental information is then provided to the Gap Analysis function, where suitable gaps in traffic are identified and evaluated.

Based on the available traffic gaps, the Planning Module computes a feasible merge trajectory, including the required speed profile and timing for the maneuver. Once a valid trajectory has been generated, the system proceeds to the Merge Execution phase, where steering and acceleration commands are issued to guide the vehicle safely into the target lane.

During execution, the system continuously evaluates the safety of the selected gap. If the gap remains available, the merge is completed successfully and the vehicle stabilizes within the target lane. However, if the gap becomes unavailable or a hazardous situation is detected, the normal merge process is interrupted and the Vehicle Control System initiates an abort procedure. Depending on the severity of the situation, emergency braking may be activated to maintain a safe vehicle state.

Following an emergency intervention, the system enters a monitoring phase in which the hazardous situation is evaluated. If conditions become safe again,

the vehicle may resume normal operation and re-enter the perception and planning cycle. In the event of a collision or an unrecoverable hazard, the Safety & Emergency System initiates emergency response procedures, including contacting emergency services.

The Activity Diagram seen in Figure 4 captures both the nominal operational behavior and the emergency handling procedures of the autonomous merge maneuver. It complements the State Machine Diagram by emphasizing process flow and subsystem responsibilities, while maintaining consistency with the functional decomposition presented in the BDD and IBD models.

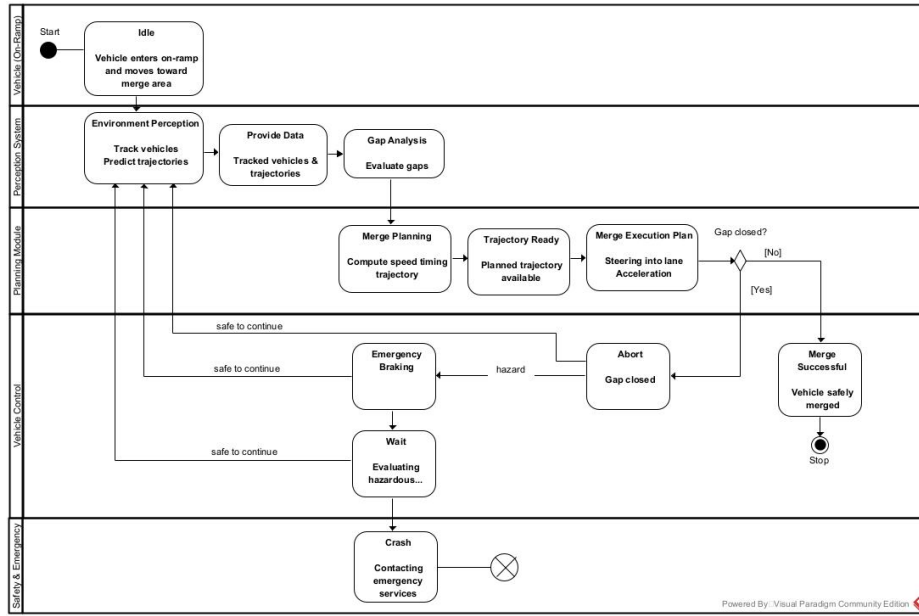


Fig. 4. Activity Diagram of the Autonomous Highway Merge Maneuver

4.2 State Machine Diagram

The state machine diagram models the behavior of the robotaxi system during a highway on-ramp merging scenario. It describes how the autonomous vehicle reacts to environmental conditions, evaluates traffic situations, and performs safe merging operations while ensuring passenger safety and collision avoidance.

Begins in the Idle state. Once the vehicle detects that it is entering an on-ramp (onRampEntering transition), the system moves to the Monitor Environment state, where it tracks surrounding vehicles within 150 m, using onboard sensors. The system constantly checks for potential collision risks. If a collision risk is detected, the system transitions to the Emergency Braking state. In the Emergency Braking state, the system overrides passenger comfort constraints to

prioritize safety. When the hazard is cleared and the road becomes safe again (hazardCleared [safeToContinue]), it returns to the monitoring state.

If no immediate danger is detected, the system evaluates the traffic conditions in the Evaluate Traffic Gap state. Here, it tries to predict movement of surrounding vehicles, calculates relative speeds, and determines whether a sufficiently large gap exists for a safe merge maneuver. When a safe gap is identified (safeGapFound), the system proceeds to the Plan Merge state. Otherwise, if no gap is available and lane distance remains (noSafeGap [remainingLane > 50m]), the system enters the Request Gap state.

The Request Gap state, the vehicle tries to communicate its intention to merge by activating turn signals and applying smooth braking to encourage neighboring vehicles to create space. After a safe gap becomes available, the system transitions back to Plan Merge.

The Plan Merge state, system computes optimal acceleration and timing needed to safely enter the lane. Once the merge plan is ready (planReady), the vehicle proceeds to the Execute Merge state, where it follows the calculated trajectory and performs the merge maneuver.

If traffic conditions suddenly become unsafe during execution (gapClosedBelowThreshold), the system aborts the maneuver and transitions to the Abort/Replan state. In this state, the vehicle cancels the current merge attempt and performs smooth braking while recalculating new plan to merge, the system returns to the merge planning phase.

When the lane merge is completed successfully (mergeCompleted), the state machine enters the Merge Successful state, indicating that the robotaxi has stabilized in the lane and the merging operation was successful.

This state machine demonstrates how the robotaxi CPS manages dynamic traffic situations through continuous monitoring, adaptive decision-making, and safety-oriented behavior. The model highlights the integration of sensing, planning, communication, and control functions necessary for reliable autonomous driving.

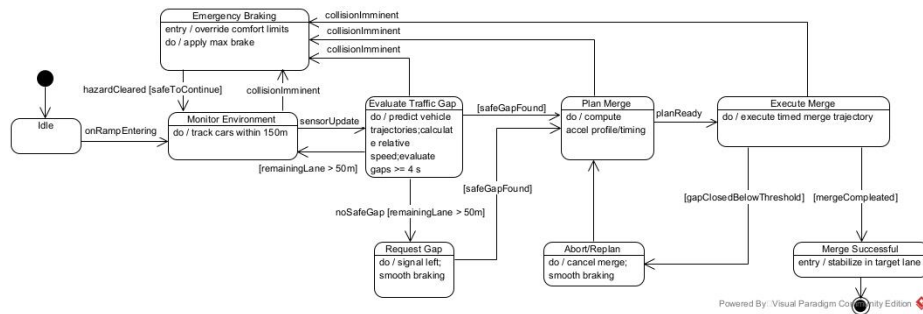


Fig. 5. State Machine Diagram

5 Verification and Validation

5.1 Modeling in UPPAAL

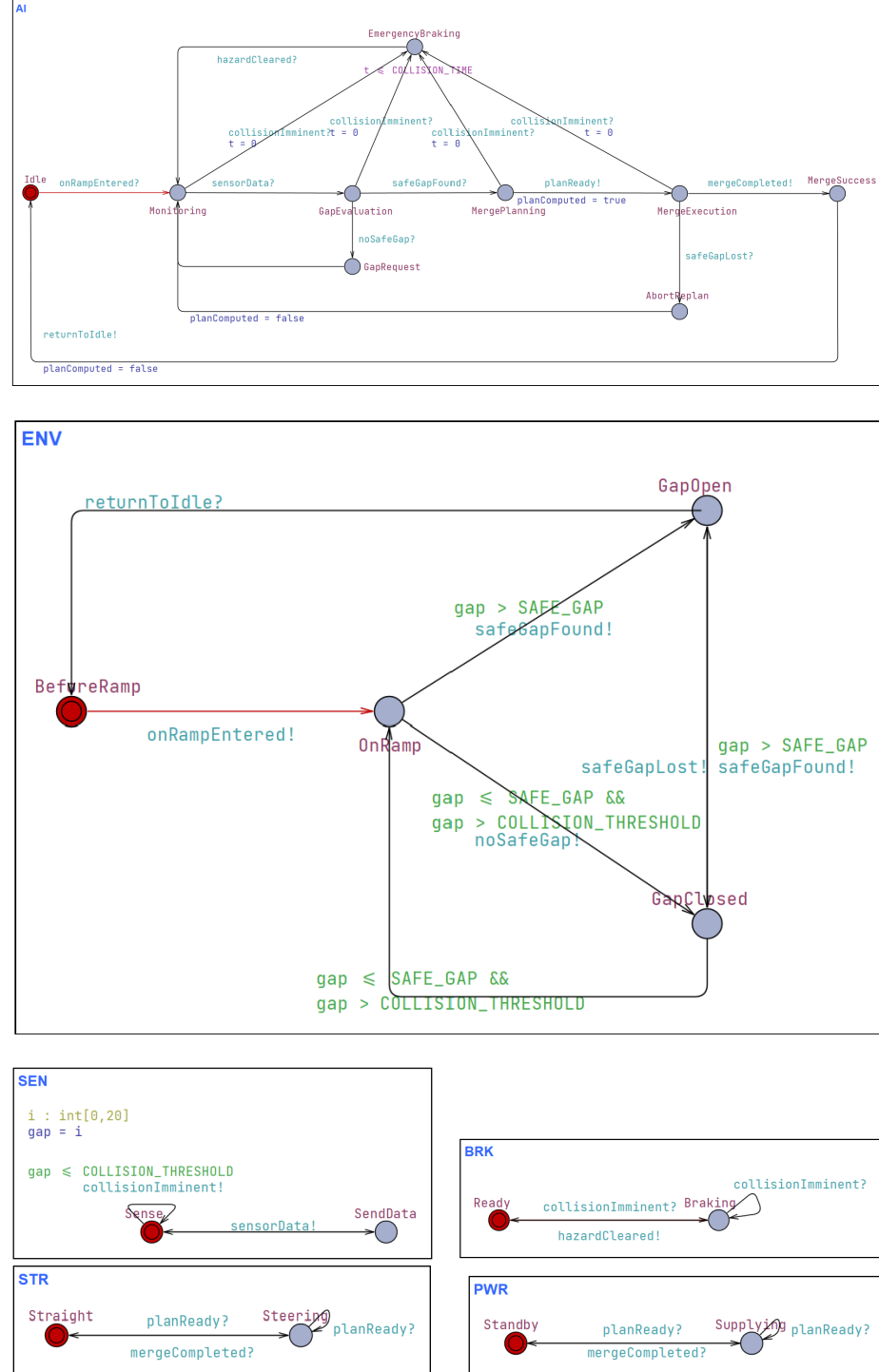


Fig. 6. UPPAAL model of the Autonomous Electric Robotaxi System Architecture

The major behaviors of the autonomous robotaxi defined in the SysML state machine diagrams were translated into UPPAAL Timed Automata (TA) models. The model consists of multiple independent templates representing functional components, while interactions among them are implemented using synchronization channels.

The system is composed of Environment, Sensor, AI Controller, Brake Controller (BRK), Steering Controller (STR), and Powertrain Controller (PWR). The Environment generates surrounding traffic conditions, while the Sensor collects environmental information and forwards it to the AI Controller. Based on the received information, the AI Controller makes decisions related to lane changing, emergency braking, and speed control.

The decisions generated by the AI Controller are transmitted to dedicated control modules. The Brake Controller is responsible for emergency braking and vehicle deceleration, the Steering Controller manages lane keeping and lane-changing maneuvers, and the Powertrain Controller controls acceleration and propulsion during normal driving.

Communication between components is modeled using UPPAAL synchronization channels. For example, when the Sensor detects a collision risk, it sends a `collisionImminent!` signal, which is received by the AI Controller through `collisionImminent?`. The AI Controller then enters the `EmergencyBraking` state and activates the Brake Controller through a `brakeCommand!` signal.

To represent timing constraints, a clock variable `t` is introduced. During emergency braking, hazardous situations must be resolved within a predefined time bound. These real-time requirements are modeled using clock constraints and state invariants.

5.2 Verification

A[] not deadlock	●
E<> AI.MergeSuccess	●
E<> AI.EmergencyBraking	●
E<> AI.GapEvaluation	●
A<> (AI.Monitoring imply AI.GapEvaluation)	●
A[] AI.MergeExecution imply gap > SAFE_GAP	●
A[] AI.MergeExecution imply planComputed	●
E<> AI.GapRequest	●
AI.MergeExecution && gap ≤ SAFE_GAP → AI.AbortReplan	●
E<> (AI.EmergencyBraking && BRK.Braking)	●
E<> (AI.MergeExecution && PWR.Supplying)	●
AI.GapEvaluation && gap > THRESHOLD → AI.MergeSuccess	●
gap ≤ COLLISION_THRESHOLD → AI.EmergencyBraking	●
A[] AI.EmergencyBraking imply AI.t ≤ COLLISION_TIME	●

Fig. 7. UPPAAL verification results for the Autonomous Electric Robotaxi System Architecture

The Autonomous Robotaxi model was verified using the UPPAAL Model Checker to evaluate safety, reachability, and timing-related requirements.

Query 1: A[] not deadlock The verification result showed that this query was satisfied.

This means that no deadlock can occur in any possible execution path of the system. It confirms that all components (AI, Sensor, BRK, STR, and PWR) can continuously interact and operate without becoming permanently blocked.

Query 2: E<> AI.MergeSuccess The verification result showed that this query was satisfied.

This indicates that there exists at least one execution path in which the vehicle can successfully complete a safe lane merge.

Query 3: E<> AI.EmergencyBraking The verification result showed that this query was satisfied.

This confirms that the AI Controller can reach the EmergencyBraking state when a collision risk is detected under certain environmental conditions.

Query 4 (MERGE-001): E<> AI.GapEvaluation Requirement MERGE-001: The system shall track vehicles in the adjacent lane.

The verification result showed that this query was satisfied.

This query verifies that the AI Controller can reach the GapEvaluation state, where information received from sensors is analyzed to determine surrounding traffic conditions and lane availability.

The result confirms that the system is capable of observing and evaluating adjacent-lane traffic as required.

Query 5 (MERGE-002): $A \langle \rangle (AI.Monitoring \text{ imply } AI.GapEvaluation)$

Requirement MERGE-002: The system shall evaluate surrounding traffic information before planning a lane merge.

The verification result showed that this query was satisfied.

This query verifies that whenever the AI Controller is monitoring the environment, the system is capable of progressing to the GapEvaluation state where sensor information is analyzed.

The GapEvaluation state represents the stage in which the AI Controller processes the detected traffic conditions and determines whether a safe lane-change opportunity exists.

The verification result confirms that the system can consistently perform traffic-gap assessment as part of the lane-change decision-making process. Therefore, the requirement for evaluating surrounding traffic information before merge planning is satisfied.

Query 6 (MERGE-003): $A[] AI.MergeExecution \text{ imply } gap > SAFE_GAP$

Requirement MERGE-003: The system shall identify a minimum safe gap.

The verification result showed that this query was satisfied.

This query verifies that lane merging is only executed when the available gap exceeds the predefined safe-gap threshold.

The result confirms that the system always maintains a safe distance before initiating a merge maneuver.

Query 7 (MERGE-004): $A[] AI.MergeExecution \text{ imply } planComputed$

Requirement MERGE-004: The system shall compute a merge plan before executing the maneuver.

The verification result showed that this query was satisfied.

This property verifies that a lane-change plan must be generated before the vehicle enters the MergeExecution state.

In the model, the variable planComputed is set to true only after the AI Controller successfully completes the MergePlanning phase. Therefore, MergeExecution can only be reached after a valid merge plan has been computed.

The verification result confirms that, for all possible execution paths, the system never performs a lane merge without first generating a valid plan.

Query 8 (MERGE-005): $E \langle \rangle AI.GapRequest$

Requirement MERGE-005: If no safe gap exists, the system shall initiate a gap request.

The verification result showed that this query was satisfied.

The result confirms that the controller can enter the GapRequest state whenever a suitable merge opportunity is unavailable.

Query 9 (MERGE-006): $\text{AI.MergeExecution} \ \&\& \ \text{gap} \leq \text{SAFE_GAP} \rightarrow \text{AI.AbortReplan}$ Requirement MERGE-006: The system shall abort the merge if the gap becomes unsafe.

The verification result showed that this query was satisfied.

The result confirms that whenever the available gap becomes unsafe during merge execution, the controller transitions to AbortReplan.

Query 10 (MERGE-007): $\text{E} \langle \rangle (\text{AI.EmergencyBraking} \ \&\& \ \text{BRK.Braking})$ Requirement MERGE-007: The system shall apply maximum braking in an emergency.

The verification result showed that this query was satisfied.

The result confirms that there exists an execution scenario in which the AI Controller enters EmergencyBraking and the Brake Controller performs braking simultaneously.

Query 11: $\text{E} \langle \rangle (\text{AI.MergeExecution} \ \&\& \ \text{PWR.Supplying})$ The verification result showed that this query was satisfied.

This query was introduced to verify that the Powertrain Controller provides propulsion while the vehicle is executing a lane-change maneuver.

In the model, once the AI Controller completes the MergePlanning phase, a planReady! signal is transmitted to the Powertrain Controller. Upon receiving this signal, the Powertrain Controller transitions to the Supplying state and provides the propulsion required for lane-changing operations.

The verification result confirms that there exists at least one execution path in which MergeExecution occurs while the Powertrain Controller is in the Supplying state. This demonstrates that the propulsion subsystem operates correctly during lane-change maneuvers.

Query 12: $\text{AI.GapEvaluation} \ \&\& \ \text{gap} > \text{THRESHOLD} \rightarrow \text{AI.MergeSuccess}$ The verification result showed that this query was not satisfied.

This query failed because the model does not guarantee that the system will always transition to the MergeSuccess state whenever the gap is sufficiently large during the GapEvaluation state.

Several situations may prevent this property from holding:

- The AI may remain in the GapEvaluation state.
- Other events may occur before the merge process is completed.
- The Environment component may generate a new gap value.
- The system may pass through MergeExecution and later return to GapEvaluation.

Therefore, even when the gap condition is satisfied, there is no formal guarantee that all execution paths will eventually reach MergeSuccess. As a result, the query fails.

Query 13: $\text{gap} \leq \text{COLLISION_THRESHOLD} \rightarrow \text{AI.EmergencyBraking}$

The verification result showed that this query was not satisfied.

This indicates that there exist execution paths in which the gap becomes smaller than the collision threshold, but the AI Controller does not immediately enter the EmergencyBraking state.

Possible reasons include:

- The Sensor has not yet transmitted the collisionImminent signal.
- The AI Controller is waiting for another event in a different state.
- Intermediate states must be traversed before reaching EmergencyBraking.

In the current design, emergency braking is not triggered directly by the gap value itself. Instead, it is activated only after a collision warning event is generated and transmitted by the Sensor component. Therefore, the condition “ $\text{gap} \leq \text{COLLISION_THRESHOLD}$ ” alone is insufficient to guarantee an immediate transition to the EmergencyBraking state, causing the query to fail.

Query 14: $\text{A}[] \text{AI.EmergencyBraking} \text{ imply } \text{AI.t} \leq \text{COLLISION_TIME}$

The verification result showed that this query was satisfied.

This means that the timing constraint is never violated while the system is in the EmergencyBraking state. Therefore, the system is capable of responding to collision threats within the specified time limit.

5.3 Validation

In addition to formal verification, the model was validated through simulation using the UPPAAL simulator. Several key scenarios were executed, including normal driving, lane changing, collision detection, and emergency braking.

When a collision risk occurred, the Sensor transmitted the collisionImminent signal, causing the AI Controller to transition into the EmergencyBraking state. Once the hazard was removed, the hazardCleared event was triggered and the system returned to normal driving operation.

In the lane-changing scenario, lane changes were only performed when a safe gap was available. If no safe gap existed, the vehicle remained in its current lane.

The simulation results were consistent with the requirements and behavioral specifications defined during the SysML design phase. Therefore, the developed UPPAAL model accurately represents the key behaviors of the autonomous robotaxi and satisfies both safety and timing-related requirements.

5.4 Results

The verification and validation activities demonstrated that the Autonomous Robotaxi model satisfies the majority of its functional, safety, and timing requirements.

Out of fourteen verification queries, twelve were successfully satisfied. The verified properties include deadlock freedom, reachability of critical operational states, correct coordination among system components, compliance with timing constraints, and safety requirements during lane-changing maneuvers.

The results confirmed that the AI Controller is capable of generating valid merge plans before executing lane changes, that successful lane merges are achievable under appropriate conditions, and that the Powertrain Controller provides propulsion during merge execution. Furthermore, emergency braking scenarios were verified, showing that the Brake Controller responds appropriately when collision risks are detected.

The timing verification results demonstrated that emergency braking operations satisfy the specified response-time constraint. In addition, the safety verification confirmed that lane changes are only executed when the available gap is greater than `SAFE_GAP`.

Two queries were not satisfied. The first query assumed that a sufficiently large gap would always lead to a successful lane merge, while the second assumed that a small gap would directly trigger emergency braking. These assumptions were not fully supported by the current system design and therefore failed during verification.

Overall, the verification and simulation results indicate that the developed UPPAAL model successfully captures the key operational behaviors of the autonomous robotaxi and satisfies the most important system requirements.

5.5 Analysis

The verification results provide evidence that the proposed system architecture operates correctly under a wide range of scenarios. The successful verification of deadlock freedom indicates that the interaction between the Environment, Sensor, AI Controller, Brake Controller, Steering Controller, and Powertrain Controller does not lead to system-wide blocking situations.

The satisfied reachability queries demonstrate that both lane-changing and emergency-braking behaviors can be executed when appropriate conditions are present. Furthermore, the successful verification of component coordination confirms that planning, propulsion, and braking functions are correctly integrated within the overall control architecture.

The failed queries reveal important characteristics and limitations of the current design. The failure of the query relating `GapEvaluation` directly to `MergeSuccess` indicates that a safe gap alone is insufficient to guarantee a completed merge. Additional environmental changes, state transitions, or decision-making processes may influence the outcome. Similarly, the failure of the collision-threshold

query shows that emergency braking is event-driven rather than directly triggered by the gap value. The Sensor component must first detect and communicate the collision risk before the AI Controller can react.

These failed properties do not necessarily represent design errors. Instead, they highlight the distinction between environmental conditions and controller decisions within the model. Such results provide valuable feedback for future refinement of both the system design and the formal requirements.

6 Discussion

The verification and validation results demonstrate that the proposed AER architecture satisfies several important safety, functional, and timing requirements. The successful verification of the deadlock-freedom property confirms that the interaction among the Environment, Sensor, AI Controller, Brake Controller, Steering Controller, and Powertrain Controller does not lead to situations where the system becomes permanently blocked. This result is particularly important for autonomous vehicles, where continuous operation and responsiveness are critical for safe driving.

The reachability properties further show that the model can successfully perform key autonomous driving functions. The successful verification of the lane-changing properties indicates that the AI Controller is capable of generating a valid merge plan and executing a lane-change maneuver when a safe gap is available. Similarly, the emergency braking properties confirm that the system can detect hazardous situations and transition into emergency braking mode when collision risks are identified.

The synchronization-based communication mechanism implemented in UP-PAAL effectively captures the interaction among the system components. Verification results demonstrate that the AI Controller correctly coordinates with the Brake Controller and Powertrain Controller through communication channels, ensuring that braking and propulsion actions are performed when required. This behavior supports the correctness of the subsystem interactions originally specified in the SysML design models.

The timing-related verification results also provide evidence that the system satisfies its real-time requirements. The successful verification of the emergency braking timing constraint shows that hazardous situations can be handled within the specified response time. This is an important aspect of safety-critical autonomous systems, where delayed reactions may lead to accidents or unsafe behavior.

Although most properties were successfully verified, two queries were not satisfied. The failure of the property relating a sufficiently large gap to guaranteed merge success indicates that the current model allows alternative execution paths in which the merge process may be interrupted or delayed. This result reflects realistic driving environments, where external conditions and dynamic traffic situations can influence the outcome of a lane-change maneuver. There-

fore, the failure of this query does not necessarily indicate a design flaw but rather highlights the non-deterministic nature of autonomous driving environments.

Similarly, the failure of the property requiring immediate emergency braking whenever the gap falls below the collision threshold reveals a limitation of the current model structure. In the implemented design, emergency braking is triggered through a collision warning signal generated by the Sensor component rather than directly by the gap value itself. Consequently, there may exist intermediate states before the AI Controller enters the EmergencyBraking state. This behavior reflects the intended architecture but also suggests that additional safety constraints could be introduced in future versions of the model to strengthen responsiveness to hazardous situations.

Overall, the combined SysML and UPPAAL modeling approach proved effective for developing and analyzing the robotaxi system. SysML provided a structured framework for defining requirements, architecture, and behavior, while UPPAAL enabled formal verification of safety and timing properties. The results demonstrate that model-based systems engineering can support the development of reliable autonomous vehicle systems by identifying design strengths, verifying critical properties, and revealing areas for further improvement before implementation.

7 Conclusion

This report demonstrated the application of Model-Based Systems Engineering (MBSE) techniques to the design, analysis, verification, and validation of an Autonomous Electric Robotaxi (AER) highway merge system. SysML was used to define the system architecture, requirements, and behavior, enabling the decomposition of a complex autonomous driving scenario into manageable subsystems with clearly defined interactions.

The developed SysML models, including requirement, structural, and behavioral diagrams, provided the foundation for creating formal UPPAAL Timed Automata models. These models enabled both simulation-based validation and formal verification of safety, reachability, and timing-related properties. Through simulation, the robotaxi behavior was evaluated under normal driving, lane-changing, and emergency braking scenarios.

The verification and validation results demonstrated that the current system design satisfies the majority of the modeled functional, safety, and timing requirements. The model was shown to be free from deadlocks, capable of executing lane-change maneuvers under suitable conditions, and able to respond appropriately to collision risks through emergency braking behavior. Timing constraints associated with safety-critical actions were also successfully verified.

Although most verification properties were satisfied, two queries were not verified. The results showed that a sufficiently large traffic gap does not guarantee successful merge completion under all execution paths and that emergency braking depends on collision-warning events rather than being triggered directly

by a gap threshold. These findings provide useful insight for future refinement of both the model and the associated requirements.

Overall, the combined use of SysML and UPPAAL proved effective for the early assessment of system safety and reliability. Future work could increase the fidelity of the AI Control Unit and environmental models, introduce more complex traffic scenarios, and explore additional fault-tolerance and verification requirements to further improve the robustness of the robotaxi system.

A Contributions

Andrej Cvecka

- Developed the State Machine Diagram for the autonomous highway merge scenario.
- Contributed to the behavioral modeling of the system.
- Authored the Abstract and Introduction sections of the report.

Yunseo Cho

- Developed the UPPAAL Timed Automata model.
- Implemented timing constraints and synchronization mechanisms.
- Designed and executed the verification queries.
- Analyzed and documented the verification and validation results.

Natatas Homrossukon

- Identified and defined the system requirements.
- Created the SysML Requirement Diagram.
- Assisted in the development of the UPPAAL model.
- Contributed to the design and evaluation of the verification queries.

Dominik Łuniewski

- Contributed to the development of the Internal Block Diagram (IBD).
- Assisted with the System Modeling chapter.
- Reviewed and proofread the report to improve clarity, consistency, and technical accuracy.

Patrick Schürpf

- Established the report structure and LNCS documentation template.
- Contributed to the System Modeling chapter.
- Developed the Block Definition Diagram (BDD) and contributed to the Internal Block Diagram (IBD).
- Authored the Discussion and Conclusion sections.
- Assisted in the integration and final editing of the report.